

TirGeo Frontend Integration Guide

Backend API contract for building the web and field UI. Generated from the current Fastify, Prisma and Docker configuration in this repository.

API base path	/api/v1
Local Docker URL	http://localhost:3000
Auth	JWT bearer token from POST /api/v1/auth/login. Send Authorization: Bearer <token>.
Primary date format	Use ISO 8601 strings. The backend coerces date fields with Zod.
Tenant boundary	Every authenticated request is scoped by organisationId embedded in the JWT.
Interactive API docs	Swagger UI is mounted at /docs when the backend is running.

Table Of Contents

1. Product scope and recommended UI modules
2. Authentication, roles, errors and shared patterns
3. Endpoint catalogue
4. HSEQ implementation guide
5. Screen-by-screen frontend build notes
6. Payload examples
7. Docker and local integration setup
8. Open items for frontend and backend alignment

1. Product Scope

TirGeo is an Australian civil construction operations API. It already covers core subcontractor workflows similar to modern construction operations platforms: projects, crews and workers, plant, pre-starts, daily reports, SWMS/JSA style safety documents, HSEQ records, timesheets, payroll export, tender review, progress claims and variations.

Recommended navigation model

- Dashboard: operational summary, open safety actions, pending documents, active permits, recent observations, overdue controls and payroll/timesheet exceptions.
- Projects: project list, lifecycle status, cost codes, linked daily reports, HSEQ records, claims and variations.
- Field app: today schedule context from project selection, daily report capture, observations, incident reporting, permit sign-on, plant pre-starts and timesheets.
- HSEQ: hazard register, controls, SWMS/JSA documents, inspections, observations, incidents, permits and corrective actions.
- Plant: asset register, published pre-start templates, pre-start submission, defects and clearance workflow.
- Timesheets and payroll: worker entry, signature, supervisor approval, correction, export configuration and export status.
- Commercial: tenders, tender document upload/review, progress claims and variations.
- Admin: users/workers, roles, pre-start templates, payroll mappings, CORS/environment settings and audit history when exposed.

Data ownership model

The UI should treat organisationId as a login-time value only. After login, do not send organisationId in normal API requests unless the endpoint explicitly asks for it. The server enforces tenancy from the JWT. Most project-bound records require a projectId and are verified against the authenticated organisation.

2. Auth, Roles And Shared Patterns

Enum	Values
Role	OWNER, ADMIN, PROJECT_MANAGER, SUPERVISOR, FOREMAN, SAFETY_MANAGER, PAYROLL, WORKER, SUBCONTRACTOR, CLIENT_AUDITOR
Status	DRAFT, SUBMITTED, APPROVED, REJECTED, ACTIVE, INACTIVE, CLOSED, CANCELLED
ProjectStatus	TENDER, AWARDED, MOBILISING, ACTIVE, ON_HOLD, PRACTICAL_COMPLETION, DEFECTS_LIABILITY, CLOSED
RiskLevel	LOW, MEDIUM, HIGH, EXTREME
InspectionResult	PASS, DEFECT, OUT_OF_SERVICE

401	Missing/invalid token or failed credential/PIN validation.
403	Authenticated user lacks the required role or is trying to access another worker's record.
404	Record not found, including records outside the organisation boundary.
409	Business rule conflict: invalid status transition, changed signed content, duplicated active export, backwards meter reading, etc.
422	Domain dependency missing, for example payroll mappings required before export.
400	Zod validation failure. The response includes code VALIDATION_ERROR and details.

The frontend should render validation details near the relevant fields, and use conflict responses for workflow-level banners such as 'refresh this timesheet before signing' or 'plant cannot be passed while a defect is present'.

3. Endpoint Catalogue

Area	Method	Path	Purpose	Auth
Health	GET	/health	Basic service liveness.	Public
Health	GET	/ready	Database readiness check.	Public
Auth	POST	/api/v1/auth/login	Login with organisationId, email and password. Returns token and user summary.	Public
Auth	PUT	/api/v1/auth/signature-pin	Create or change a four digit signing PIN.	JWT
Projects	GET	/api/v1/projects	List projects for the organisation.	JWT
Projects	POST	/api/v1/projects	Create project.	OWNER, ADMIN, PROJECT_MANAGER
Projects	PATCH	/api/v1/projects/:id/status	Move project through the allowed lifecycle.	OWNER, ADMIN, PROJECT_MANAGER
Field	POST	/api/v1/field/daily-reports	Create site diary/daily report with personnel, plant, activities, quantities, delays, visitors and photos.	JWT
Plant	GET	/api/v1/plant	List plant assets.	JWT
Plant	POST	/api/v1/plant	Create plant asset.	OWNER, ADMIN, PROJECT_MANAGER
Plant	GET	/api/v1/plant/pre-start-templates	List published templates for field use, optionally filtered by plantType.	JWT
Plant	GET	/api/v1/plant/pre-start-templates/manage	List all templates for management.	OWNER, ADMIN
Plant	POST	/api/v1/plant/pre-start-templates	Create a draft pre-start template version.	OWNER, ADMIN
Plant	PATCH	/api/v1/plant/pre-start-templates/:id	Edit a draft template.	OWNER, ADMIN
Plant	POST	/api/v1/plant/pre-start-templates/:id/publish	Publish a draft template.	OWNER, ADMIN
Plant	POST	/api/v1/plant/:id/pre-starts	Submit plant pre-start answers, defects and signature.	JWT
Plant	POST	/api/v1/plant/:id/clearance	Clear defect/out-of-service plant.	OWNER, ADMIN, PROJECT_MANAGER, SAFETY_MANAGER
Safety	POST	/api/v1/safety/documents	Create SWMS, JSA, toolbox, risk, environmental, traffic or emergency plan.	Safety managers and project leaders
Safety	POST	/api/v1/safety/documents/:id/approve	Approve draft safety document and lock content hash.	OWNER, ADMIN, SAFETY_MANAGER
Safety	POST	/api/v1/safety/documents/:id/publish	Assign approved document to workers.	Safety managers and project leaders
Safety	POST	/api/v1/safety/documents/:id/acknowledge	Worker signs assigned/published document.	JWT worker assignment
Safety	GET	/api/v1/safety/my-assignments	Current worker's safety document assignments and acknowledgements.	JWT
Safety	POST	/api/v1/safety/incidents	Create incident report.	JWT
HSEQ	GET	/api/v1/safety/hazards	List hazard register with controls/actions. Filters: projectId, status, domain, riskLevel.	JWT
HSEQ	POST	/api/v1/safety/hazards	Create hazard with optional controls.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN, SAFETY_MANAGER
HSEQ	PATCH	/api/v1/safety/hazards/:id/status	Update hazard status, residual risk and review date.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN, SAFETY_MANAGER
HSEQ	POST	/api/v1/safety/hazards/:id/controls	Create control measure under hazard.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN, SAFETY_MANAGER
HSEQ	GET	/api/v1/safety/controls	List controls. Filters: projectId, type, dueOnly.	JWT

Area	Method	Path	Purpose	Auth
HSEQ	PATCH	/api/v1/safety/controls/:id/verify	Mark a control as verified.	OWNER, ADMIN, PM, SUPERVISOR, SAFETY_MANAGER
HSEQ	GET	/api/v1/safety/observations	List safety/HSEQ observations. Filters: projectId, type, status.	JWT
HSEQ	POST	/api/v1/safety/observations	Create hazard, near miss, unsafe act, positive, environmental or quality observation.	JWT
HSEQ	GET	/api/v1/safety/inspections	List HSEQ inspections with items/actions. Filters: projectId, type, status.	JWT
HSEQ	POST	/api/v1/safety/inspections	Create inspection with checklist item results.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN, SAFETY_MANAGER
HSEQ	PATCH	/api/v1/safety/inspections/:id/complete	Close inspection with score/result.	OWNER, ADMIN, PM, SUPERVISOR, SAFETY_MANAGER
HSEQ	GET	/api/v1/safety/permits	List permits to work. Filters: projectId, type, status.	JWT
HSEQ	POST	/api/v1/safety/permits	Create permit to work.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN, SAFETY_MANAGER
HSEQ	PATCH	/api/v1/safety/permits/:id/status	Move permit status and set approver/closedAt when relevant.	OWNER, ADMIN, PM, SUPERVISOR, SAFETY_MANAGER
HSEQ	GET	/api/v1/safety/actions	List corrective actions. Filters: projectId, ownerId, status, overdue.	JWT
HSEQ	POST	/api/v1/safety/actions	Create corrective action linked to incident, hazard, observation, inspection or permit.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN, SAFETY_MANAGER
HSEQ	PATCH	/api/v1/safety/actions/:id	Update owner, due date, priority, status and closure notes.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN, SAFETY_MANAGER
HSEQ	GET	/api/v1/safety/dashboard	Counts and recent records for HSEQ dashboard. Optional projectId.	JWT
Timesheets	GET	/api/v1/timesheets/approvers	List eligible approvers.	JWT
Timesheets	POST	/api/v1/timesheets	Create draft timesheet with entries.	JWT
Timesheets	POST	/api/v1/timesheets/:id/submit	Employee signs and submits to approver.	JWT worker owner
Timesheets	POST	/api/v1/timesheets/:id/approve	Approver signs submitted timesheet.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN
Timesheets	POST	/api/v1/timesheets/:id/onsite-approve	Shared-device supervisor PIN approval.	JWT
Timesheets	POST	/api/v1/timesheets/:id/reject	Reject submitted timesheet with reason.	OWNER, ADMIN, PM, SUPERVISOR, FOREMAN
Timesheets	POST	/api/v1/timesheets/:id/correct	Create correction from rejected timesheet.	JWT worker owner
Payroll	PUT	/api/v1/payroll/connections/:provider	Configure XERO or MYOB connection metadata.	OWNER, ADMIN, PAYROLL
Payroll	PUT	/api/v1/payroll/connections/:provider/employees/:workerId	Map local worker to external payroll employee.	OWNER, ADMIN, PAYROLL
Payroll	PUT	/api/v1/payroll/connections/:provider/pay-items/:localCode	Map local pay/allowance code to external pay item.	OWNER, ADMIN, PAYROLL
Payroll	POST	/api/v1/payroll/exports	Build payroll export from approved timesheets.	OWNER, ADMIN, PAYROLL
Payroll	PATCH	/api/v1/payroll/exports/:id/status	Mark export SENT, RECONCILED or FAILED.	OWNER, ADMIN, PAYROLL

Area	Method	Path	Purpose	Auth
Commercial	POST	/api/v1/commercial/tenders	Create tender.	OWNER, ADMIN, PROJECT_MANAGER
Commercial	GET	/api/v1/commercial/tenders/:id	Tender detail with documents, requirements and checklist.	OWNER, ADMIN, PROJECT_MANAGER
Commercial	POST	/api/v1/commercial/tenders/:id/documents	Upload and analyse tender document multipart file.	OWNER, ADMIN, PROJECT_MANAGER
Commercial	PATCH	/api/v1/commercial/tenders/:tenderId/requirements/:id	Confirm or reject suggested requirement.	OWNER, ADMIN, PROJECT_MANAGER
Commercial	PATCH	/api/v1/commercial/tenders/:tenderId/checklist/:id	Update tender checklist owner/status/due date.	OWNER, ADMIN, PROJECT_MANAGER
Commercial	POST	/api/v1/commercial/progress-claims	Create progress claim.	OWNER, ADMIN, PROJECT_MANAGER
Commercial	POST	/api/v1/commercial/variations	Create variation.	OWNER, ADMIN, PROJECT_MANAGER
Sync	POST	/api/v1/sync/register	Register offline-capable device.	JWT
Sync	GET	/api/v1/sync/pull	Pull audit event cursor stream.	JWT
Sync	POST	/api/v1/sync/receipt	Store client mutation receipt.	JWT
Notifications	GET	/api/v1/notifications	List latest 100 notifications.	JWT
Notifications	POST	/api/v1/notifications/:id/read	Mark notification read.	JWT

4. HSEQ Implementation Guide

Enum	Values
HseqDomain	HEALTH, SAFETY, ENVIRONMENT, QUALITY
HazardStatus	IDENTIFIED, ASSESSED, CONTROLLED, CLOSED
ControlType	ELIMINATION, SUBSTITUTION, ISOLATION, ENGINEERING, ADMINISTRATIVE, PPE
HseqInspectionType	SITE_WALK, PRE_START, ENVIRONMENTAL, QUALITY_AUDIT, HSEQ_AUDIT, PLANT
SafetyObservationType	HAZARD, NEAR_MISS, UNSAFE_ACT, POSITIVE_BEHAVIOUR, ENVIRONMENTAL, QUALITY
PermitType	HOT_WORK, CONFINED_SPACE, EXCAVATION, WORKING_AT_HEIGHT, LIFTING, ELECTRICAL_ISOLATION, TRAFFIC_CONTROL, ENVIRONMENTAL, OTHER
PermitStatus	DRAFT, REQUESTED, APPROVED, ACTIVE, SUSPENDED, CLOSED, CANCELLED

HSEQ dashboard cards

- Open hazards: /api/v1/safety/dashboard.openHazards
- High risk hazards: highRiskHazards, count of HIGH and EXTREME non-closed hazards.
- Overdue controls: overdueControls, unverified controls with reviewDueAt before now.
- Open and overdue actions: openActions and overdueActions.
- Open incidents: openIncidents.
- Active permits: activePermits, permits APPROVED or ACTIVE and not expired.
- Pending documents: pendingDocuments for DRAFT or SUBMITTED safety docs.
- Recent activity: recentObservations and recentInspections arrays.

Core HSEQ workflows

- Observation to action: user reports observation, supervisor reviews, creates corrective action linked by observationId, closes action with completion notes/evidence.
- Hazard to controls: safety manager creates hazard register item, adds controls using the hierarchy of controls, verifies controls, updates hazard residual risk, then closes when controlled.
- Inspection to action: supervisor creates inspection with item results. Failed/defect items should prompt action creation linked by inspectionId.
- Permit lifecycle: create permit in DRAFT, progress through REQUESTED, APPROVED, ACTIVE, SUSPENDED or CLOSED. Use status endpoint for approvals and closure.
- Document lifecycle: draft SWMS/JSA, approve to generate content hash, publish to workers, workers acknowledge with signature. Display hash/approved date in UI for defensible records.

5. Screen Build Notes

Screen	Build notes
Login	Requires organisationId, email and password. Store token securely. Keep user.role for client-side navigation only; server remains authoritative.
Project list	Fetch GET /projects. Use status chips and jurisdiction. Create/edit status only for OWNER, ADMIN and PROJECT_MANAGER.
Field daily report	Project picker, report date, weather, personnel rows, plant rows, activities, quantities, delays, visitors, safety notes and photo IDs.
Hazard register	Table with risk, residual risk, domain, status, review due, controls count and actions count. Provide quick filters for project, status, domain and risk.
Control verification	Show controls due or overdue. Capture effectiveness, evidenceDocumentId and next reviewDueAt.
Observation capture	Mobile-first form with type, project, location, risk, description, immediate action and photos.

Screen	Build notes
Inspection builder	Use sections and questions client-side. Submit items with PASS, DEFECT or OUT_OF_SERVICE. Show action-required toggles.
Permit board	Filter by type/status. Warn when expiresAt is close. For active permits, surface location, scope, hazards, controls and signOns.
Corrective actions	Kanban or table grouped by status/overdue. Actions can link back to incident, hazard, observation, inspection or permit.
Safety documents	Author document, approve, publish assignments, worker acknowledgement. UI must prevent signing before approval/publish.
Plant pre-start	Fetch template by plantType, render questions, validate required fields client-side, require defect details for failed checks.
Timesheets	Draft entry grid, submit with signature, approver queue, approve/reject, correction flow. Preserve signatures as base64/string payloads.
Payroll	Configure provider, map workers/pay items, create export from approved timesheets, update export status.
Commercial	Tender upload uses multipart field file. Show extracted requirements and generated checklist for review.

6. Payload Examples

Login

```
POST /api/v1/auth/login
{
  "organisationId": "00000000-0000-0000-0000-000000000000",
  "email": "admin@tirgeo.local",
  "password": "ChangeMe123!"
}
```

Create hazard with controls

```
POST /api/v1/safety/hazards
Authorization: Bearer <token>
{
  "projectId": "<projectId>",
  "title": "Live services in excavation zone",
  "description": "Potential contact with electrical or comms services during trenching.",
  "domain": "SAFETY",
  "activity": "Excavation",
  "location": "Chainage 120-180",
  "riskLevel": "HIGH",
  "residualRiskLevel": "MEDIUM",
  "reviewDueAt": "2026-08-14T00:00:00.000Z",
  "controls": [
    {
      "type": "ENGINEERING",
      "title": "Pothole services",
      "description": "Verify all service locations before mechanical excavation.",
      "verificationMethod": "Supervisor inspection"
    },
    {
      "type": "ADMINISTRATIVE",
      "title": "Permit and spotter",
      "description": "Permit to excavate and dedicated spotter required."
    }
  ]
}
```

Create HSEQ observation

```
POST /api/v1/safety/observations
{
  "projectId": "<projectId>",
  "type": "NEAR_MISS",
  "title": "Reversing exclusion zone breach",
  "description": "Worker crossed behind reversing loader before spotter stopped movement.",
  "location": "Northern stockpile",
  "riskLevel": "HIGH",
  "immediateAction": "Stopped work, reset exclusion zone and briefed crew.",
  "photos": ["photo-storage-id-1"]
}
```

Create inspection

```
POST /api/v1/safety/inspections
{
  "projectId": "<projectId>",
  "type": "SITE_WALK",
  "title": "Morning site walk",
  "location": "Stage 2",
  "status": "SUBMITTED",
  "items": [
    {
      "section": "Access",
      "question": "Walkways clear and signed",
      "result": "PASS"
    },
    {
      "section": "Excavation",
      "question": "Edge protection in place",
      "result": "DEFECT",
      "notes": "Missing barrier at east side",
      "correctiveActionRequired": true
    }
  ]
}
```



```
    }  
  ]  
}
```

Create permit

POST /api/v1/safety/permits

```
{  
  "projectId": "<projectId>",  
  "type": "EXCAVATION",  
  "title": "Excavate stormwater trench",  
  "location": "Lot 12 frontage",  
  "scope": "Open trench from pit A to pit B.",  
  "startsAt": "2026-07-15T22:00:00.000Z",  
  "expiresAt": "2026-07-16T06:00:00.000Z",  
  "hazards": [{"title": "Live services", "riskLevel": "HIGH"}],  
  "controls": [{"title": "Dial before you dig reviewed"}, {"title": "Spotter in place"}],  
  "signOns": []  
}
```

Create corrective action

POST /api/v1/safety/actions

```
{  
  "projectId": "<projectId>",  
  "description": "Install barrier on east side excavation edge.",  
  "ownerId": "<workerId>",  
  "dueAt": "2026-07-16T00:00:00.000Z",  
  "priority": "HIGH",  
  "inspectionId": "<inspectionId>",  
  "source": "Site walk defect"  
}
```

7. Docker And Local Setup

The repository now includes a Dockerfile that builds the TypeScript app, generates Prisma client code, and starts by applying migrations with `prisma migrate deploy` before running the Fastify server. docker-compose.yml now starts both Postgres and the API.

```
# Build and run API plus Postgres
docker compose up --build

# API
http://localhost:3000

# Swagger UI
http://localhost:3000/docs

# Health checks
GET http://localhost:3000/health
GET http://localhost:3000/ready
```

Environment variables

DATABASE_URL	Postgres connection string. Compose uses postgresql://tirgeo:tirgeo@postgres:5432/tirgeo.
JWT_SECRET	Required. Must be at least 32 characters. Replace the compose placeholder before shared environments.
PORT	Defaults to 3000.
HOST	Defaults to 0.0.0.0.
STORAGE_PATH	Upload storage path. Compose maps /app/data/uploads to a named Docker volume.
CORS_ORIGINS	Comma-separated allowed frontend origins. Update for deployed frontend URLs.
TRUST_PROXY	Set true when behind a trusted reverse proxy/load balancer.

Seed data

The seed script creates a demo organisation and owner login, plus a generic plant pre-start template. After containers are running, run this if a fresh local database needs demo access:

```
docker compose exec api npm run db:seed
```

The seed output prints organisationId, email and password. Use those values in POST /api/v1/auth/login.

8. Open Alignment Items

- Worker and user management endpoints are not currently exposed. The frontend will need these before a complete admin UI can create workers, assign roles, and map worker user accounts.
- Photo/document upload is implemented for tender documents only. Daily reports, HSEQ photos, evidenceDocumentId and attachments currently expect identifiers rather than a general media upload endpoint.
- The HSEQ action creation endpoint accepts source IDs but does not yet validate that the linked source belongs to the same project. The UI should only present valid in-project choices.
- Safety document list/detail endpoints are limited. The UI can create/approve/publish/acknowledge and fetch my assignments, but management list/detail screens may need additional GET endpoints.
- Swagger is available for live exploration, but richer OpenAPI schemas may require adding explicit Fastify schema metadata if the frontend team wants generated clients.
- Offline sync currently streams audit event metadata, not full entity snapshots. Treat it as a foundation for sync rather than a complete offline replication API.